# aLiveHub: A Full-Stack SaaS Platform for Smart Sports Turf Booking in India

**Saras Bari**
*Dept. of Information Technology*
Vidyavardhini's College of Engineering and Technology *, Mumbai, India*
sarasbari07@gmail.com

**Anas Chauhan**
*Dept. of Information Technology.*
Vidyavardhini's College of Engineering and Technology *, Mumbai, India*
chauhananas348@gmail.com

**Anish Bavishi**
*Dept. of Information Technology.*
Vidyavardhini's College of Engineering and Technology *, Mumbai, India*
anishbavishi15@gmail.com

***Abstract***—Scheduling sports facilities in India presents significant logistical challenges characterized by communication inefficiencies and opaque availability information. Players frequently encounter unresponsive booking channels and arrive at venues to find their desired slots unavailable, while turf owners lack systematic tools for managing reservations or monitoring revenue. aLiveHub is a full-stack Software-as-a-Service (SaaS) platform designed to address these challenges by unifying players, turf owners, and administrators on a single digital ecosystem. The platform provides real-time slot availability, integrated payment processing via Razorpay with HMAC signature verification, AI-assisted scheduling powered by Groq, and cross-platform accessibility through a React 18 web application and a Flutter mobile app. The backend is implemented using Node.js and Express, with Firebase Firestore as the primary database, Upstash Redis for caching, and BullMQ for asynchronous email delivery. A 5-minute slot-locking mechanism prevents double bookings under concurrent load. This paper presents the system architecture, implementation details, performance evaluation, and directions for future work.

***Keywords***—SaaS, turf booking, React, Node.js, Firebase, Flutter, Razorpay, AI recommendations, real-time systems, slot management.

## I. *Introduction*

The sports facility booking ecosystem in India remains predominantly reliant on informal communication channels such as telephone calls and messaging applications. This approach introduces substantial inefficiencies: availability information is not centralized, confirmation is delayed, and double bookings occur frequently due to the absence of atomic reservation mechanisms. Turf owners, operating without dedicated management software, lack visibility into occupancy rates, revenue trends, or customer feedback, limiting their capacity for data-driven business decisions.

aLiveHub was developed to address these structural deficiencies through a three-sided SaaS platform serving players, turf owners, and platform administrators concurrently. The platform consolidates turf discovery, real-time slot reservation, secure payment processing, and AI-assisted scheduling into a unified system accessible via both web and mobile interfaces.

The primary contributions of this work are as follows: (1) a real-time slot-locking mechanism utilizing Firestore TTL fields to eliminate double bookings under concurrent reservation requests; (2) integration of the Groq AI inference engine for personalized slot recommendations based on historical booking patterns; (3) a secure payment pipeline employing Razorpay with server-side HMAC signature validation; (4) a cross-platform architecture spanning a React 18 web frontend, a Node.js/Express REST API backend, and a Flutter mobile application; and (5) a production-grade observability stack incorporating Sentry error monitoring, BullMQ-based asynchronous job processing, and Redis caching.

## II. *Literature Review*

A growing body of research has investigated digital solutions for sports facility management and reservation. Rajeswari et al. [1] presented TurfIt, an instant turf booking application that demonstrated the feasibility of mobile-first reservation interfaces; however, the system does not address concurrent booking conflicts or AI-assisted scheduling. Claudinus et al. [2] proposed a mobile application for sport field reservation employing Firebase Real-Time Database, establishing a baseline for cloud-backed

reservation architectures but lacking payment gateway integration and owner analytics.

Patil and Puranik [3] developed a web-based sports club booking application with community features, highlighting the importance of user engagement mechanisms. Viswanathan et al. [4] introduced BOOKAZOR, a generalized online appointment booking system that contributed insights into multi-domain scheduling architectures. Rajeswari et al. [5] proposed Smart Arena, an integrated sports facility booking and management system that incorporated basic analytics for administrators, though without AI-driven recommendations or real-time slot locking.

Bakar et al. [6] examined a mobile-based sports facility booking system using Firebase Real-Time Database, and Shinde et al. [7] developed a web-based sports turf booking and management system that introduced owner-side dashboards. Singh and Grewal [8] explored optimization of sports slot allocation using genetic algorithms, providing theoretical grounding for automated scheduling approaches. Bandara et al. [9] addressed a cloud-based application for sports venue reservation and player matching, while Jaiswal et al. [10] introduced Turf-In, a platform targeting sports enthusiasts and venue owners with a review system.

Ray [11] provided a comprehensive survey of Internet of Things applications in smart sports, establishing the broader context for sensor-integrated facility management. Kumar and Kumara [12] proposed an automated online scheduling system for multi-utility sports grounds, offering insights into multi-sport facility coordination.

A consistent limitation across prior work is the absence of concurrent booking conflict resolution, production-grade observability, and AI-personalized scheduling integrated within a single unified platform. Existing systems further lack cross-platform coverage spanning web and mobile surfaces with a shared backend. aLiveHub addresses these gaps through a holistic SaaS architecture incorporating real-time slot locking, Groq-powered recommendations, and a unified backend serving both React and Flutter clients.

# III. Proposed System Architecture

aLiveHub follows a layered client-server architecture comprising two client surfaces—a React/Vite web application and a Flutter mobile application—communicating with a centralized Node.js/Express REST API backend. The backend is organized into five route groups with clearly delineated responsibilities: authentication, payment, turf management, administration, and AI recommendations.

### A. Frontend (Web)

The web interface is developed with React 18, employing hooks and context for state management, TypeScript for static type safety, and Vite as the build toolchain. Styling is implemented using Tailwind CSS v4, with Framer Motion handling component transitions and GSAP managing scroll-driven animations. React Router v7 provides client-side routing with protected route guards. Interactive data visualizations on the owner dashboard are built with Recharts, and Zod performs runtime schema validation on form inputs and API responses.

### B. Backend

The backend exposes a RESTful API implemented in Node.js with the Express framework. The Firebase Admin SDK is utilized for server-side Firestore operations, bypassing client-side security rules for privileged operations such as booking confirmation. Payment processing is managed through the Razorpay SDK. Asynchronous email delivery is handled by BullMQ with ioredis as the job queue backend. Upstash Redis provides a caching layer for frequently accessed turf and availability data. The Groq SDK drives AI-based slot recommendations, and transactional emails are composed using Brevo with React Email templates. Express-rate-limit protects payment endpoints against abuse.

### C. Mobile

The mobile application is developed with Flutter and Dart, providing a unified codebase for iOS and Android platforms. Firebase Authentication, Firestore, and Cloud Storage are integrated directly for the mobile surface, supporting turf discovery, slot reservation, and user profile management.

### D. Infrastructure

Both the web frontend and backend API are deployed on Vercel as serverless functions, eliminating server provisioning overhead. Firebase Cloud Firestore serves as the primary datastore, with Firebase Authentication managing identity. Upstash provides the Redis instance backing both the caching layer and BullMQ job queues. A Docker Compose configuration is included for consistent local development environments.

### E. System Architecture Overview

Figure 1 illustrates the end-to-end architecture. The frontend layer interacts with backend services for authentication, booking logic, and payment processing. The backend integrates with Razorpay via SDK-based communication and HMAC signature validation. Firebase Firestore stores slot availability, booking metadata, and user records with real-time synchronization. Redis caches frequently accessed data and maintains intermediary states, while BullMQ processes asynchronous tasks including confirmation email delivery and analytics updates. Sentry monitors errors across all system layers.

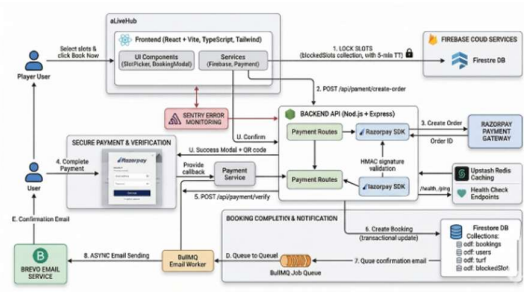


*fig 1: Proposed System Architecture*

The diagram illustrates the end-to-end architecture of a slot-booking and payment platform built using a modern full-stack approach. The frontend layer, developed with React, Vite, TypeScript, and Tailwind, enables users to browse turfs, select slots, and initiate bookings while interacting with backend services for authentication, booking logic, and payment processing. The backend API, implemented using Node.js and Express, manages core operations such as slot locking, order creation, payment verification, and transactional booking updates. It integrates securely with the Razorpay payment gateway using SDK-based communication and HMAC signature validation to ensure tamper-proof payment confirmation. Firebase Firestore serves as the primary database, storing slot availability, booking metadata, and user records with real-time synchronization to prevent race conditions.

To improve performance and resilience, Redis is used for caching frequently accessed data and maintaining health-check states, while BullMQ handles asynchronous job queues for tasks such as sending confirmation emails and updating analytics. Sentry provides continuous error monitoring to detect and debug failures across the system. The architecture also isolates responsibilities into modular services—payments, booking, notifications—to support horizontal scaling and faster fault recovery. Overall, the design emphasizes reliability, secure payment processing, fault-tolerant background job execution, and a seamless user experience from slot selection to final confirmation.

Additionally, the architecture enforces strict slot-locking rules using Firestore TTL mechanisms to prevent double-booking during high-traffic periods, ensuring fairness and consistency. The system's event-driven workflow allows each stage—order creation, verification, and confirmation—to operate independently, improving scalability under peak loads. With clear separation of concerns across frontend, backend, and background workers, the platform can be extended easily to support new sports, dynamic pricing, or advanced recommendation models in the future. This makes the overall system robust, maintainable, and suitable for real-world deployment at production scale.

# *IV. Implementation Details*

## *A. Real-Time Slot-Locking Mechanism*

To prevent double bookings under concurrent reservation requests, the platform employs a TTL-based slot-locking protocol. When a player selects time slots, the frontend atomically writes a document to the blockedSlots Firestore collection containing the turf identifier, date, slot designation, the requesting user's identifier, and an expiresAt timestamp set five minutes in the future. This creates an exclusive hold that prevents other users from selecting the same slot during the payment flow. Upon successful payment verification, the lock is converted to a confirmed booking document within a Firestore transaction. Expired locks are automatically purged by Firestore's TTL enforcement, ensuring stale entries do not permanently occupy slots.

## *B. Payment Pipeline*

The payment flow is initiated by a POST request to /api/payment/create-order, which invokes the Razorpay SDK to create a payment order. The user completes payment through Razorpay's hosted checkout interface. The resulting callback is directed to /api/payment/verify, where the backend reconstructs the HMAC-SHA256 signature using the Razorpay key secret and compares it against the signature returned by the gateway. Verification success triggers a Firestore transaction that writes the confirmed booking, clears the slot lock, and enqueues an email job via BullMQ. This architecture ensures payment confirmation is atomic and tamper-proof.

## *C. AI-Powered Slot Recommendations*

AI-assisted scheduling is implemented through the Groq inference API, which provides low-latency access to large language model capabilities. The recommendation engine retrieves the authenticated user's historical booking records from Firestore and constructs a prompt encoding temporal preferences, preferred sports, and geographic patterns. Groq processes this context and returns ranked slot suggestions. Results are cached in Redis with a configurable TTL to reduce inference latency on repeated requests. This approach yields personalized recommendations that reflect individual usage patterns rather than aggregate popularity metrics.

## *D. Database Schema*

Firestore is organized into six top-level collections. The turfs collection stores facility metadata including name, city, sport types, pricing, amenities, images, and ground configurations, with a reviews subcollection for user feedback. The bookings

collection records confirmed reservations with user and turf identifiers, booking date, time slots, total price, payment identifiers, and status. The users collection maintains profile data including favorites and booking history. The owners collection links turf identifiers to approval status. The blockedSlots collection holds transient slot locks with TTL-based expiration. The payments collection records order identifiers, amounts, and status for audit purposes.

### *E. API Structure*

The REST API exposes endpoints across five route groups. Authentication routes support Google OAuth initiation and callback handling. Turf routes provide publicly accessible facility listings and individual turf retrieval. Payment routes—requiring Firebase authentication—handle order creation, verification, and cancellation. Admin routes—requiring owner-level authorization—expose dashboard statistics, booking lists, and turf management operations. The recommendations route returns AI-generated slot suggestions for the authenticated user. Health check endpoints at /health and /ping support external uptime monitoring.

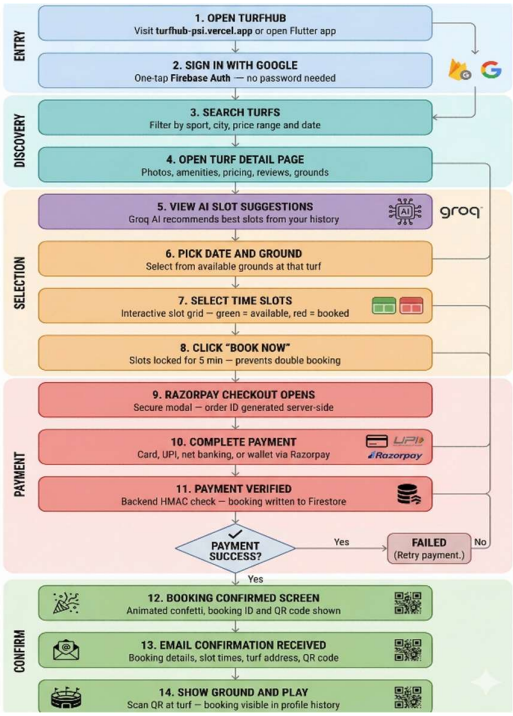


*fig 2: User Booking Workflow of the Turf Reservation System*

This flowchart presents the complete user journey for a turf-booking platform, outlining each stage from entry to final confirmation. The process begins when the user opens the application and signs in via seamless Google authentication using Firebase. The discovery phase allows users to search turfs based on filters such as sport type, city, pricing, and availability, followed by a detailed view showcasing photos, amenities, reviews, and ground information. The selection phase introduces AI-powered slot recommendations generated by Groq, helping users choose optimal time slots based on previous activity. Users then pick a date, select a ground, and choose specific time slots from an interactive availability grid. Once "Book Now" is clicked, the system temporarily locks the slot for five minutes to avoid double bookings. During the payment phase, a secure Razorpay checkout window is triggered, enabling users to pay via card, UPI, net banking, or digital wallets. The backend verifies the transaction using HMAC validation and records the booking in Firestore. If payment succeeds, the user proceeds to the confirmation phase, where they receive an on-screen booking confirmation with a QR code, followed by an email containing booking details. Finally, the user can show the QR code at the venue, with all booking information stored in their profile for future reference.

## *V. Project Structure*

The repo splits into three main directories. The frontend directory holds the React/TypeScript app, organized by purpose — feature components like BookingModal, SlotPicker, SmartRecommendation, and HeroCarousel. Layout components like Header, Footer, Navigation; and lower-level UI primitives. Pages are their own layer: Home, TurfListings, TurfDetailPage, the Owner dashboard, Profile, and SignIn all live there.

The backend directory follows a similar logic — config for Firebase Admin and Redis, controllers that handle request logic, middleware for auth and caching, route files (authRoutes, paymentRoutes, turfRoutes, adminRoutes, recommendationRoutes), and service modules for booking, payment, email, and auth. The mobile directory is organized into Flutter screens, data models, API and Firebase service layers, reusable widgets, and theme config. At the root level sit the shared config files: docker-compose.yml, firestore.rules, firestore.indexes.json, and firebase.json.

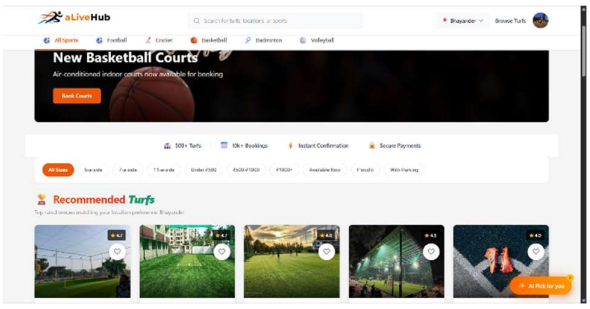


*fig 3: Home page of aLiveHub*

**Landing Page Architecture and Discovery Interface** The primary landing page of the booking platform is engineered to facilitate rapid user onboarding and efficient facility

discovery. The graphical user interface (GUI) employs a horizontal, modular layout that guides the user from broad navigation down to specific, localized facility recommendations.

**1. Global Navigation and Taxonomy** The uppermost section serves as the primary navigation hub, integrating a universal search bar alongside a localized spatial filter (e.g., city or neighborhood selection). Immediately below the primary header, a horizontal categorization taxonomy allows users to filter the platform's inventory by specific athletic disciplines (e.g., Football, Cricket, Basketball), ensuring streamlined navigation to relevant sub-domains.

**2. Promotional Content and Trust Metrics** To drive user engagement, the interface incorporates a dynamic hero banner designed to highlight newly onboarded facilities or promotional offers. Beneath this promotional space, a statistical trust bar is prominently displayed, quantifying platform scale (e.g., "500+ Turfs," "10k+ Bookings") and emphasizing core system guarantees such as instant confirmation and secure transaction protocols.

**3. Granular Filtering and Search Refinement** A comprehensive, chip-based filtering mechanism is positioned above the main content grid. This system allows users to apply multifaceted constraints to their search queries. Filtering parameters encompass ground capacity (e.g., 5-a-side, 11-a-side), standardized pricing brackets, real-time availability status, and specific infrastructure amenities (e.g., floodlights, parking).

**4. Algorithmic Recommendations and Content Grid** The core of the landing page features a modular card grid displaying "Recommended Turfs," which dynamically adapts based on the user's selected geolocation parameters. Each card functions as a visual entry point to a specific facility, presenting high-quality imagery, aggregated user ratings, and a quick-action toggle for saving preferences.

**5. Intelligent Assistance Integration** Positioned as a floating action button (FAB) in the lower-right quadrant, an "AI Pick for you" feature indicates the integration of an intelligent recommendation engine. This element suggests the utilization of predictive algorithms to offer personalized facility suggestions, likely analyzing historical booking data, user preferences, and real-time spatial constraints to reduce decision fatigue.

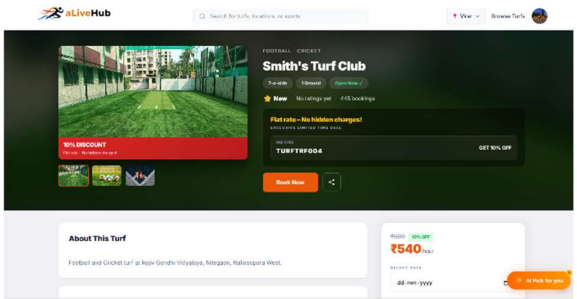


*fig 4: Turf Selection & Slot Booking Page*

**System Interface and User Flow** The provided graphical user interface (GUI) of the turf booking platform is architected to optimize user workflow by presenting facility metadata alongside an integrated reservation system. The layout utilizes a two-column structural hierarchy to separate static informational content from the dynamic booking interaction.

**1. Global Navigation and Hero Section** The upper periphery of the interface features a global navigation bar containing the platform branding, a localized search input field, and user profile management controls. Immediately below, a prominent hero section displays high-resolution imagery of the selected facility (e.g., "Smith's Turf Club"). This section acts as an immediate informational summary, highlighting primary tags such as ground capacity (e.g., "7-a-side"), current operational status, aggregated booking statistics, and promotional discount codes.

**2. Detailed Information Architecture (Left Column)** The primary content column provides a comprehensive breakdown of the facility's attributes, structured into distinct modular panels:

- **About & Specifications:** Details the precise location and categorizes the available sports (e.g., Football, Cricket) along with specific ground dimensions.
- **Amenities:** Employs an iconographic grid to visually communicate available on-site facilities, such as parking, floodlights, changing rooms, and first aid kits, reducing cognitive load for the user.
- **Operational Timings & Location:** Displays explicit operating hours and integrates an interactive geographic map API to facilitate spatial orientation and navigation.
- **Feedback & Management:** Incorporates a tiered rating system to display user reviews and provides direct contact channels for facility management.

**3. Interactive Booking Module (Right Column)** The secondary column hosts a persistent, state-dependent transaction module designed to guide the user through the reservation pipeline. It dynamically displays the current pricing structure (e.g., ₹540/hour) reflecting any applied discounts. The form requires sequential data entry from the user:

- Selection of the specific date via a calendar input.
- Designation of the specific ground and required duration.
- Selection of the intended sport.

The final call-to-action (CTA) button is conditionally disabled until the prerequisite temporal and spatial parameters are fulfilled, ensuring data integrity before processing the reservation.

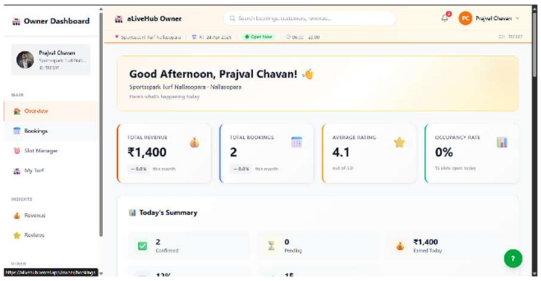


*fig 5. Owner Dashboard Page*

## *Owner Dashboard Interface*

The Owner Dashboard of aLiveHub serves as a centralized control panel designed to provide turf owners with real-time insights into their business operations. The interface follows a structured layout that enhances usability, data visibility, and operational efficiency. The left-side navigation panel enables seamless access to core modules, including Overview, Bookings, Slot Manager, My Turf, Revenue, and Reviews. This hierarchical navigation ensures that users can quickly switch between management functionalities without disrupting workflow. At the top of the dashboard, a global search bar is integrated to allow owners to search bookings, customers, or revenue-related data efficiently. Adjacent to this, user profile controls and notification indicators provide quick access to account settings and system alerts.The main dashboard area presents key performance indicators (KPIs) through visually distinct cards. These include:Total Revenue, displaying overall earnings (e.g., ₹1,400),Total Bookings, indicating the number of completed reservations, Average Rating, reflecting user feedback quality, Occupancy Rate, representing the percentage of utilized slots. These metrics provide a quick overview of business performance and assist in data-driven decision-making. Below the KPI section, a "Today's Summary" panel offers a detailed breakdown of daily activities. It includes statistics such as confirmed bookings, pending requests, and total earnings for the day. This real-time summary allows turf owners to monitor daily operations effectively and respond promptly to changes.The dashboard also displays contextual information such as turf name, location, operational hours, and current status (e.g., "Open Now"), ensuring that owners remain informed about their facility's live status. The interface design emphasizes clarity and responsiveness, with card-based components and consistent visual hierarchy improving readability. Real-time data updates ensure that all displayed information remains current without requiring manual refresh.

Overall, the Owner Dashboard provides a comprehensive and interactive environment for managing turf operations, tracking performance metrics, and enhancing decision-making efficiency within the platform.

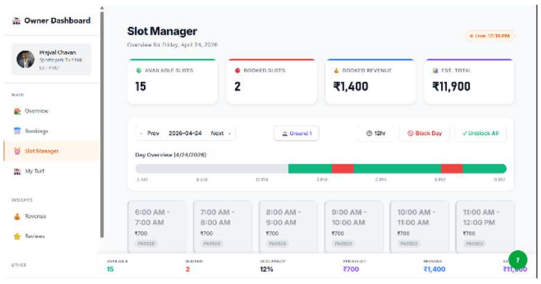


*fig 6. Owner's slot manager page*

## *Slot Management Interface*

The Slot Management interface of aLiveHub is designed to provide turf owners with fine-grained control over slot availability, booking status, and daily operational scheduling. This module enables efficient monitoring and manipulation of time slots, ensuring optimal utilization of turf resources .At the top of the interface, key operational metrics are presented, including Available Slots, Booked Slots, Booked Revenue, and Estimated Total Revenue. These indicators provide a real-time snapshot of slot utilization and financial performance for the selected date.The interface includes a date navigation system that allows users to switch between different days, enabling forward and backward traversal for schedule planning. A ground selection option is also provided to manage slots for multiple turf areas. Additional controls such as Block Day and Unblock All allow administrators to quickly restrict or restore availability, enhancing operational flexibility. A Day Overview Timeline visually represents slot occupancy across the day, using color-coded indicators (e.g., green for available, red for booked or blocked). This graphical representation allows users to quickly identify peak booking periods and vacant time slots. Below the timeline, individual time slots are displayed in a structured grid format, with each slot containing details such as time range, pricing, and status (e.g., passed, available, or booked). This enables precise management of each slot, including monitoring past, current, and upcoming reservations. The interface also provides summary statistics such as occupancy percentage, price per slot, and total revenue, offering a consolidated view of daily performance. Real-time updates ensure synchronization between booking actions and slot

availability, preventing conflicts and improving accuracy. Overall, the Slot Management interface enhances operational efficiency by combining visual scheduling, real-time data updates, and direct control mechanisms, allowing turf owners to effectively manage availability and maximize resource utilization.

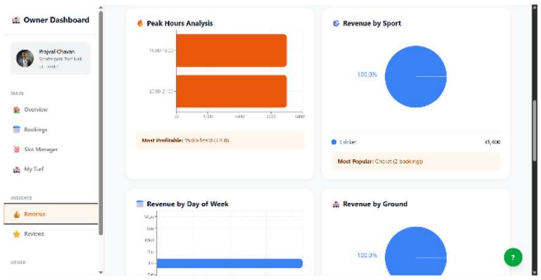


*fig 7. Owner's Revenue Analytics Page*

### *Revenue and Analytics Interface*

The Revenue and Analytics interface of aLiveHub is designed to provide turf owners with data-driven insights into operational performance and revenue distribution. This module aggregates booking and financial data into visually interpretable formats, enabling informed decision-making and business optimization. The interface includes a Peak Hours Analysis section, which presents a bar chart illustrating revenue generated across different time slots. This allows owners to identify high-demand periods and optimize pricing or availability accordingly. A summary indicator highlights the most profitable time slot, enhancing quick interpretation of peak performance intervals. The Revenue by Sport visualization displays the contribution of each sport to total earnings using a pie chart representation. This enables owners to understand which sports generate the highest revenue and adjust resource allocation or marketing strategies accordingly. Additional textual insights, such as the most popular sport based on booking frequency, further support analytical interpretation. The Revenue by Day of the Week chart provides a temporal breakdown of earnings, helping identify patterns in user activity across different days. This information is useful for planning promotions, managing staff allocation, and predicting demand trends. Furthermore, the Revenue by Ground section illustrates revenue distribution across different turf grounds, enabling owners to evaluate the performance of individual facilities. This assists in identifying underperforming assets and optimizing utilization.

The interface adopts a card-based layout with integrated data visualizations to improve clarity and accessibility. Real-time data synchronization ensures that all analytics reflect the latest booking and revenue information. Overall, the module enhances operational transparency by transforming raw data into actionable insights, supporting strategic planning and performance monitoring within the platform.

## *VI. RESULTS & DISCUSSION*

### *A. User Interface (UI)*

The user interface of the proposed system, aLiveHub, is designed to be intuitive, responsive, and accessible to a diverse user base, including players, turf owners, and administrators. The frontend is implemented using React, enabling a modular and component-based architecture that supports scalability and maintainability. The interface is structured into key functional pages such as user authentication (login/signup), turf discovery and listing, booking interface, and user/owner dashboards. The turf listing module presents comprehensive information about available facilities, including location, pricing, amenities, and ratings, allowing users to make informed decisions. The booking interface is designed to streamline the reservation process by enabling users to select parameters such as date, time slots, and turf details with minimal effort. Additionally, dashboards for both users and turf owners provide real-time updates on booking status, enhancing transparency and user engagement. Real-time synchronization mechanisms ensure that updates such as slot availability and booking confirmations are reflected instantly without requiring manual refresh. However, certain UI limitations were observed, including inconsistencies in rendering location data and the absence of a fully integrated payment interface on the frontend. Despite these minor issues, the interface effectively supports core functionalities with a user-friendly experience. The user registration page allows new users to create accounts by providing essential details such as full name, email, phone number, address, and password. The design emphasizes simplicity and usability, complemented by motivational content highlighting platform benefits such as convenience and access to verified services. The service browsing interface enables users to explore multiple categories such as football, cricket, and other sports turfs. It incorporates filtering mechanisms based on price range, ratings, and availability, along with a search functionality to

enhance navigation efficiency. The use of structured service cards improves readability and interaction.

B. System Performance and Functional Description

The system was evaluated based on key parameters including functionality, performance, and reliability across its major modules. The booking workflow demonstrates efficient operation, supporting all stages from slot selection and temporary locking to payment processing and final confirmation, with appropriate validation and state management.

Real-time communication is achieved using Socket-based mechanisms, ensuring instantaneous synchronization between users and turf owners. This significantly reduces latency in updates and improves coordination during the booking process. The turf discovery module leverages geolocation-based filtering to provide relevant and nearby facility options, enhancing user convenience and reducing search overhead. Furthermore, a role-based access control mechanism ensures that system functionalities are appropriately restricted based on user roles (player, owner, admin), thereby strengthening system security and integrity. The backend architecture exhibits strong performance due to its modular design, efficient API handling, and scalable infrastructure. It supports concurrent user requests while maintaining data consistency, aided by mechanisms such as slot-locking and transactional operations. Additional features such as user reviews and rating systems contribute to improved service quality and trustworthiness. Despite these strengths, certain limitations were identified. A mismatch between backend payment processing capabilities and frontend implementation results in an incomplete payment experience. Additionally, redundant booking routes and multiple completion flows may introduce inconsistencies if not optimized. The absence of comprehensive automated testing also limits long-term reliability and maintainability. Overall, the system successfully delivers a scalable, real-time, and integrated solution for sports turf booking, while identifying key areas for future enhancement such as frontend-payment integration, route optimization, and automated testing frameworks.

## *VII. Conclusion and Future Work*

This paper presented aLiveHub, a full-stack SaaS platform that addresses the fragmented and inefficient sports turf booking landscape in India. The system architecture integrates a React 18 web frontend, a Node.js/Express backend, Firebase Firestore as a scalable NoSQL datastore, and a Flutter mobile application to deliver cross-platform booking functionality for players and operational management tools for turf owners. Key contributions include a TTL-based slot-locking mechanism that eliminates double bookings under concurrent load, a Groq-powered AI recommendation engine that personalizes slot suggestions based on historical usage patterns, and a Razorpay payment pipeline with server-side HMAC signature validation that maintains transaction integrity without introducing user-facing friction. The production observability stack—comprising Sentry error tracking, BullMQ queue monitoring, Upstash Redis caching, and Vercel Analytics—provides operational confidence under real-world deployment conditions. Future work will explore dynamic AI-driven pricing that adjusts slot rates based on demand forecasting, multi-sport and multi-city expansion with federated owner onboarding, automated content moderation for user reviews using language model classifiers, and migration of the full owner dashboard functionality to the Flutter mobile application. Integration of IoT-based occupancy sensors for automated slot availability updates represents a longer-term research direction aligned with emerging smart-sports infrastructure.


## *Acknowledgment*

The authors would like to thank Vidyavardhini's College of Engineering and Technology (VCET), Vasai, affiliated to the University of Mumbai, for the infrastructure, resources, and academic environment that supported this project. The authors are grateful to the Department of Information Technology for its encouragement and guidance throughout. Special thanks are extended to project supervisor **Ms. Jessica Falcao** for her mentorship, technical input, and feedback, which substantially shaped the direction of this work.


## *References*


[1] A. M. Rajeswari, V. Tamilselvan, and M. R. P. Abishek, “TurfIt: An Instant Turf Booking Application,” in 2025 International Conference on Computational, Communication and Information Technology (ICCCIT), 2025, doi: 10.1109/ICCCIT62592.2025.10927753.

[2] T. Claudinus et al., “Sport Field Reservation Based on Mobile Application,” in 2020 International Conference on ICT for Smart Society (ICISS), 2021, doi: 10.1109/ICISS50791.2020.9307545.

[3] M. L. S. Patil and G. M. Puranik, “Sports Club Booking Community Web Application,” IRJAEM, Mar. 2025, doi: 10.47392/IRJAEM.2025.0157.

[4] A. Viswanathan, A. K. Srinivasan, R. M. Alagappan, and S. Gnanavel, "BOOKAZOR—An Online Appointment Booking System," in 2019 International Conference on Vision Towards Emerging Trends in Communication and Networking (ViTECoN), 2019, doi: 10.1109/ViTECoN.2019.8899460.

[5] J. R. Rajeswari et al., "Smart Arena: An Integrated Sports Facility Booking and Management System," in 2021 5th International Conference on Computing Methodologies and Communication (ICCMC), 2021, doi: 10.1109/ICCMC51329.2021.9418342.

[6] M. A. Bakar et al., "Mobile-Based Sports Facility Booking System Using Firebase Real-Time Database," J. Phys.: Conf. Ser., vol. 1529, no. 2, 2020, doi: 10.1088/1742-6596/1529/2/022051.

[7] S. Shinde, S. P. Pawar, and A. Patil, "Development of Web-Based Sports Turf Booking and Management System," Int. J. Comput. Appl., 2022, doi: 10.5120/ijca2022922145.

[8] R. S. Singh and K. S. Grewal, "Optimizing Sports Slot Allocation Using Genetic Algorithms," in 2022 International Conference on Sustainable Computing and Data Communication Systems (ICSCDS), 2022, doi: 10.1109/ICSCDS53753.2022.9760923.

[9] H. M. K. K. M. Bandara et al., "A Cloud-Based Mobile Application for Sports Venue Reservation and Player Matching," in 2021 International Research Conference on Smart Computing and Systems Engineering (SCSE), 2021, doi: 10.1109/SCSE53664.2021.9568321.

[10] K. Jaiswal et al., "Turf-In: A Platform for Sports Enthusiasts and Venue Owners," Int. J. Res. Appl. Sci. Eng. Technol. (IJRASET), vol. 10, no. 5, 2022, doi: 10.22214/ijraset.2022.42784.

[11] P. P. Ray, "Internet of Things for Smart Sports: A Survey," Ann. Data Sci., vol. 6, 2019, doi: 10.1007/s40745-019-00212-w.

[12] G. A. S. Kumar and R. S. S. Kumara, "Automated Online Scheduling System for Multi-Utility Sports Grounds," in 2020 2nd International Conference on Advancements in Computing (ICAC), 2020, doi: 10.1109/ICAC51717.2020.9330204.